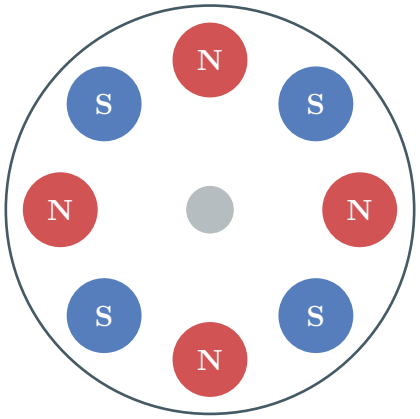
Arquitectura del software

Organización del código, patrones de diseño, pruebas y guía de uso del modelo, el CAD y el visualizador



Documento 4 de 5 · Ingeniería de software del proyecto

Basado en M. N. O. Sadiku, *Elementos de Electromagnetismo*, 3.ª ed., Oxford University Press.
Valores numéricos generados por el paquete `generador_axial` (`uv run generador-axial reporte`).

Índice

1. Objetivo y alcance **2**

2. Estructura de directorios **2**

3. Arquitectura en capas (Python) **3**
3.1. Dominio . . . 3
3.2. Aplicación . . . 4
3.3. Infraestructura . . . 4
3.4. Presentación . . . 4

4. Patrones de diseño aplicados **5**
4.1. Template Method en las gráficas . . . 5
4.2. Command en la interfaz de línea de comandos . . . 5
4.3. Despacho por tipo en el CAD . . . 6
4.4. Observer en el visualizador . . . 6

5. El visualizador 3D **6**

6. Principios de código limpio aplicados **7**

7. Pruebas automáticas **7**

8. Cómo usar el proyecto **8**

9. Cómo extender el proyecto **9**

10. Registro de correcciones **10**

1 Objetivo y alcance

El proyecto tiene tres productos de software que describen el mismo generador:

1. **Modelo analítico** (Python): calcula campo, flujo, tensión, potencia y par, y genera las gráficas.
2. **Generador CAD** (Python): produce las piezas imprimibles en STL a partir de las mismas cotas.
3. **Visualizador 3D** (JavaScript + Three.js): animación interactiva con osciloscopio y métricas.

A estos se suma la **documentación** (estos PDF), cuyos números se generan desde el modelo.

El problema central que resuelve la arquitectura es la **coherencia**: antes, cada producto tenía sus propias constantes y fórmulas, y divergían (por ejemplo, el CAD, la simulación y la guía no coincidían en las cotas ni en los resultados). Ahora existe **una sola definición del diseño** y todo se deriva de ella.

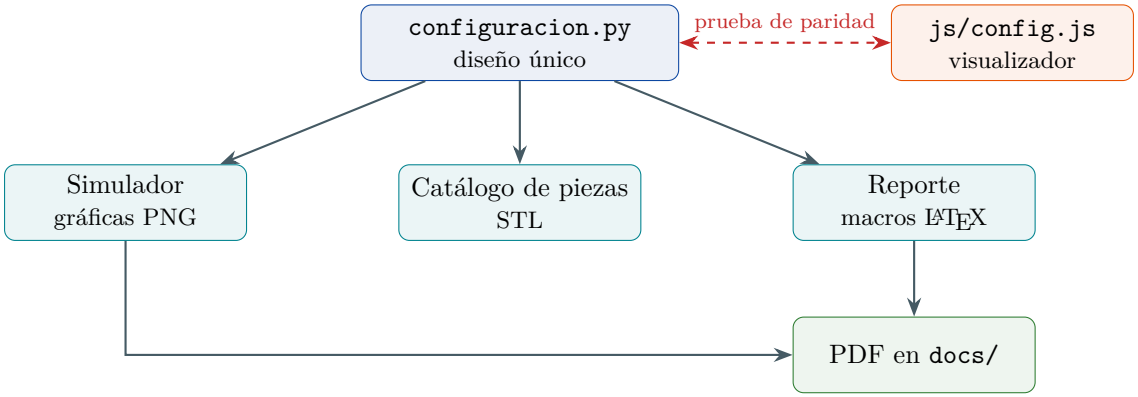


Figura 1: Fuente única de verdad. El visualizador no puede importar Python, así que replica la configuración en JavaScript y una prueba automática garantiza que ambas sean idénticas.

2 Estructura de directorios

```
teoria_electromagnetica/
|-- pyproject.toml              # proyecto uv: dependencias, comando y pytest
|-- src/generador_axial/
|   |-- configuracion.py        # diseño por defecto y escenario de análisis
|   |-- rutas.py                # directorios de salida
|   |-- dominio/                # entidades y leyes físicas (sin E/S)
|   |   |-- errores.py  componentes.py  mecanica.py  diseno.py  fisica.py
|   |-- aplicacion/             # casos de uso
|   |   |-- simulador.py  piezas.py  reporte.py
|   |-- infraestructura/        # adaptadores a formatos externos
|   |   |-- cad/  (malla.py  constructor.py  exportador.py)
|   |   |-- graficas/  (base.py  catalogo.py)
|   |   |-- latex.py
|   |-- presentacion/           # interfaz de línea de comandos
|       |-- cli.py  consola.py
|-- tests/                      # pruebas pytest (138 casos, 29 de interfaz)
|-- simulacion/modelo_analitico.py   # envoltorio: generador-axial simular
|-- simulacion/graficas/        # PNG generados
|-- cad_3d/generar_modelos_stl.py    # envoltorio: generador-axial cad
|-- cad_3d/stl/                 # STL generados
|-- visualizador_3d/
|   |-- index.html  server.py  css/style.css
```

```
|   |-- js/config.js  js/app.js
|   |-- js/modelo/  (fisica.js  dinamica.js  estado.js)
|   |-- js/vista/   (escena.js  materiales.js  osciloscopio.js  panel.js)
|   |-- js/controlador/controles.js
|   |-- js/vendor/  (three.min.js  OrbitControls.js)
|-- docs/
    |-- *.md  *.pdf  compilar_pdf.sh
    |-- latex/  (fuentes .tex, comun/preambulo.tex, generado/)
```

3 Arquitectura en capas (Python)

El paquete `generador_axial` sigue una **arquitectura limpia**: las dependencias apuntan siempre hacia el dominio. El dominio no sabe que existen matplotlib, STL ni LaTeX; por eso se puede probar de forma aislada y reutilizar en cualquier interfaz.

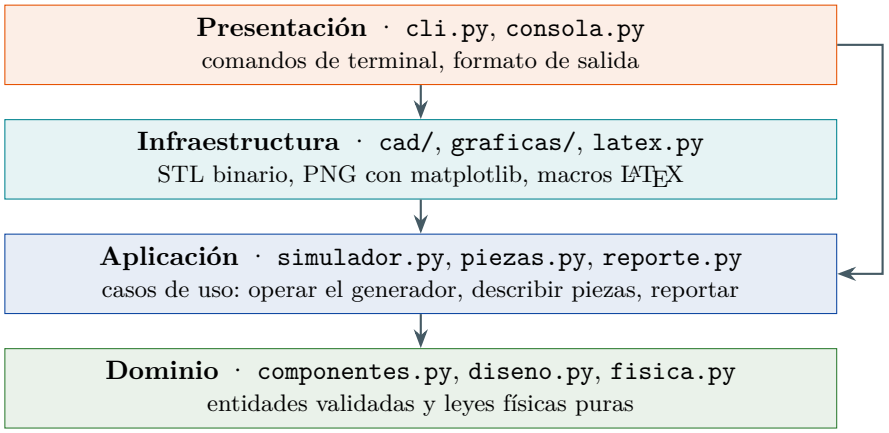


Figura 2: Capas y sentido de las dependencias (una capa solo importa a las de abajo).

3.1 Dominio

- `componentes.py`: entidades inmutables (`@dataclass(frozen=True)`) que representan cada componente físico: `ImanPermanente`, `AlambreMagneto`, `Bobina`, `Rotor`, `Estator`, `Rectificador`, `IndicadorLed` y, para la micro-red, `Bateria`, `ControladorCarga` e `InversorAc`. Cada una valida sus invariantes al construirse (por ejemplo, número de polos par, imanes que no se solapan, ocupación de la ventana de la bobina menor que 1) y lanza `ErrorDeDiseno` si no se cumplen.
- `mecanica.py`: cotas de las piezas fabricables (`GeometriaMecanica`).
- `diseno.py`: el agregado `DisenoGenerador`, que une los componentes y valida reglas que involucran a varios (tantas bobinas como polos, mismo radio de montaje, piezas que caben en los discos).
- `fisica.py`: funciones puras y vectorizadas con NumPy (campo axial, campo medio, frecuencia, fem, rectificación, rizado, corriente en carga, potencia máxima extraíble y corriente de carga de la batería). Cada función cita la ecuación del Documento 1.

```python
@dataclass(frozen=True)
class Bobina:
    espiras: int
    diametro_interior_m: float
    diametro_exterior_m: float
    altura_m: float
    alambre: AlambreMagneto
```

```
    @property
    def resistencia_ohm(self) -> float:
        return self.alambre.resistividad_ohm_m * self.longitud_alambre_m / self.alambre.
    area_cobre_m2

    @property
    def factor_ocupacion(self) -> float:
        """Fracción de la ventana ocupada si cada espira requiere un cuadrado de lado d_esmaltado
    ."""
        return self.espiras * self.alambre.diametro_esmaltado_m**2 / self.area_ventana_m2
```

3.2 Aplicación

- `simulador.py`: `SimuladorGenerador` es una *fachada* que aplica la física a un diseño concreto. Ofrece los métodos `punto_de_operacion(rpm)`, `forma_de_onda(...)` y `operar_con_carga(...)`, que devuelven objetos de valor inmutables (`PuntoDeOperacion`, `FormaDeOnda` y `OperacionConCarga`). Los valores que no cambian (campo, flujo) se calculan una vez con `cached_property`. Para la micro-red ofrece `balance_microred(rpm,potencia_carga_ac_w,soc)`, `rpm_de_equilibrio(p)`, `simular_bateria(...)` (integración del SoC con corte y reconexión por histéresis) y `forma_de_onda_inversor(...)`, que devuelven `BalanceMicrored`, `EvolucionBateria` y `SalidaInversor`.
- `piezas.py`: `catalogo_de_piezas(diseno)` es una *fábrica* que traduce el diseño a especificaciones geométricas neutras (`PlacaConAlojamientos`, `SolidoDeRevolucion`), sin saber nada del formato STL.
- `reporte.py`: `generar_reporte` recorre los pasos de la memoria de cálculo y produce una lista de `Magnitud` (clave, valor formateado, unidad) con todos los valores intermedios.

3.3 Infraestructura

- `cad/constructor.py`: convierte cada especificación geométrica en una `Malla` triangular cerrada. Los sólidos de revolución se barren 360°; las placas se triangulan con Delaunay y se descartan los triángulos cuyo centroide cae dentro de un agujero.
- `cad/malla.py`: `Malla` sabe unir mallas, calcular normales, su volumen con signo y si es hermética (cada arista compartida exactamente por dos triángulos con orientación opuesta).
- `cad/exportador.py`: escribe STL binario con un `dtype` estructurado de NumPy.
- `graficas/`: una clase por gráfica sobre la base `Grafica`.
- `latex.py`: escribe `valores.tex` (macros `\Val...`) y `tabla_ensayos.tex`.

3.4 Presentación

`cli.py` define el comando `generador-axial` con cuatro subcomandos: `simular`, `cad`, `reporte` y `todo`. `consola.py` da formato al resumen que se imprime.

4 Patrones de diseño aplicados

Tabla 1: Patrones, dónde están y qué problema resuelven.

Patrón	Ubicación	Problema que resuelve
Fuente única de verdad	`configuracion.py`, `js/config.js`	Simulación, CAD, documentos y visualizador usan las mismas cifras.
Objeto de valor inmutable	`dominio/componentes.py`	Un diseño válido no puede volverse inválido después de construirse.
Agregado	`dominio/diseno.py`	Las reglas que cruzan componentes se validan en un solo lugar.
Fachada	`SimuladorGenerador`	Oculta las fórmulas tras operaciones de alto nivel.
Fábrica	`catalogo_de_piezas`	Deriva las piezas del diseño; agregar una pieza no toca el exportador.
Despacho por tipo (*singledispatch*)	`construir_malla`	Un constructor por tipo de geometría sin cadenas de `if`.
Template Method	`graficas/base.py`	Crear, dibujar y guardar es común; cada gráfica solo implementa `dibujar`.
Command	`presentacion/cli.py`	Cada subcomando es un objeto; `todo` los compone.
MVC	`visualizador_3d/js/`	Separa cálculo (modelo), dibujo (vista) y eventos (controlador).
Observer	`js/modelo/estado.js`	Las vistas reaccionan solo a las claves del estado que cambian.
Raíz de composición	`js/app.js`	Un único lugar crea y conecta todos los objetos.

4.1 Template Method en las gráficas

```python
class Grafica(ABC):
    nombre_archivo: str

    def exportar(self, directorio: Path) -> Path:
        directorio.mkdir(parents=True, exist_ok=True)
        figura = Figure(figsize=self.tamano_pulgadas, dpi=self.resolucion_dpi, layout="tight")
        self.dibujar(figura)                    # paso variable
        ruta = directorio / self.nombre_archivo
        figura.savefig(ruta)
        return ruta

    @abstractmethod
    def dibujar(self, figura: Figure) -> None: ...
```

Se usa `matplotlib.figure.Figure` directamente (no `pyplot`) para no depender de estado global ni de un entorno gráfico: las gráficas se generan igual en un servidor sin pantalla.

4.2 Command en la interfaz de línea de comandos

```python
class Comando(ABC):
    nombre: str
    ayuda: str

    def configurar(self, parser): ...

    @abstractmethod
```

```
    def ejecutar(self, argumentos, simulador): ...

def construir_comandos() -> list[Comando]:
    basicos = [ComandoSimular(), ComandoCad(), ComandoReporte()]
    return [*basicos, ComandoTodo(basicos)]
```

Agregar un subcomando consiste en escribir una clase y añadirla a la lista; `main` no cambia.

4.3 Despacho por tipo en el CAD

```
@singledispatch
def construir_malla(geometria, longitud_arista_mm=1.0) -> Malla:
    raise TypeError(f"No hay constructor de malla para {type(geometria).__name__}.")

@construir_malla.register
def _(geometria: SolidoDeRevolucion, longitud_arista_mm=1.0) -> Malla: ...

@construir_malla.register
def _(geometria: PlacaConAlojamientos, longitud_arista_mm=1.0) -> Malla: ...
```

4.4 Observer en el visualizador

```
actualizar(cambios) {
  const cambiadas = Object.keys(cambios).filter((clave) => this._datos[clave] !== cambios[clave])
    ;
  if (cambiadas.length === 0) return;
  Object.assign(this._datos, cambios);
  const instantanea = this.instantanea();
  this._oyentes.forEach((oyente) => oyente(instantanea, new Set(cambiadas)));
}
```

Por ejemplo, la escena solo reconstruye las líneas de campo cuando cambia `entrehierroMm`, no en cada cuadro.

5 El visualizador 3D

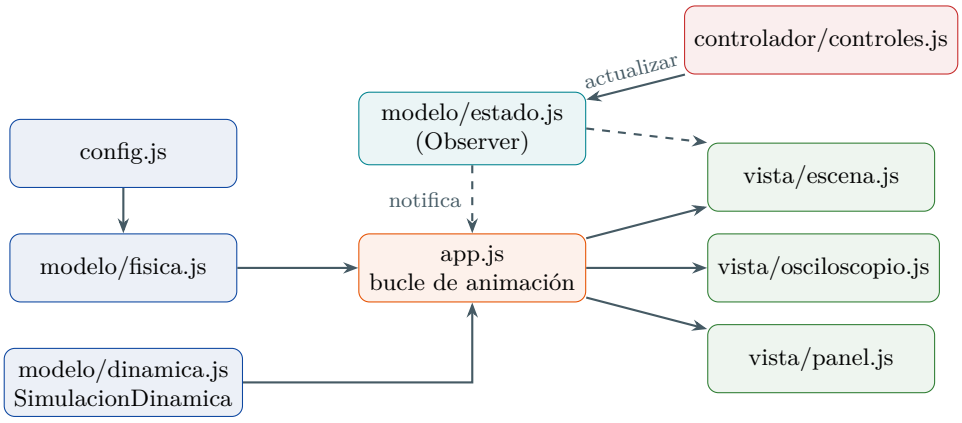


Figura 3: Módulos del visualizador. Los controles nunca dibujan: solo modifican el estado.

- **Módulos UMD**: `config.js`, `modelo/fisica.js` y `modelo/dinamica.js` funcionan tanto en el navegador (como `window.GeneradorAxial`) como en Node (`module.exports`); así la prueba de paridad los ejecuta sin navegador.

- **Sin dependencias de red**: Three.js y OrbitControls están en `js/vendor/` y se cargan con etiquetas `<script>` clásicas, de modo que `index.html` se puede abrir directamente desde el disco (`file://`) o con el servidor incluido.
- **Simulación dinámica**: `modelo/dinamica.js` (clase `SimulacionDinamica`, espejo exacto de la de Python, Documento 5) integra el rotor, la batería y la temperatura. El usuario fija el **par motriz**; la velocidad resulta del equilibrio con el rozamiento y el frenado de Lenz. El selector de tiempo (1×, 10×, 50×, 500×) acelera solo las variables lentas; en pausa el tiempo simulado se congela.
- **Osciloscopio**: muestra una ventana de 0.1 s de tiempo simulado y submuestrea cada cuadro para que la onda no sufra *aliasing* a altas RPM; CH1 dibuja la forma de onda real (no senoidal) del solape imán–bobina. Tiene dos canales: CH1 es el generador y CH2 la salida del inversor, con escala de 50, 100 o 200 V/div según la tensión elegida; si el inversor está apagado o cortado muestra "APAGADO".
- **Estado físico fuera del Observer**: velocidad, SoC y temperatura cambian en cada cuadro, así que viven en `SimulacionDinamica` y no en el estado observable (evita notificar a todas las vistas 60 veces por segundo); la aplicación pasa cada instante al panel, a la escena y al osciloscopio.
- **Render físicamente basado**: `vista/materiales.js` genera un mapa de entorno de estudio (PMREM) y texturas procedurales (contrachapado, capas de impresión, bobinado esmaltado, etiquetas); la escena usa tono ACES, salida sRGB y sombras suaves.

6 Principios de código limpio aplicados

Nombres del dominio
Las variables llevan el nombre físico y la unidad: `entrehierro_m`, `tension_pico_v`, `campo_medio_bobina_t`. Esto elimina comentarios del tipo "# en metros" y evita errores de unidades.

Unidades SI dentro, milímetros solo en los bordes
Todo el dominio trabaja en SI; la conversión a milímetros ocurre solo al generar STL o al mostrar valores.

Sin números mágicos
Cada constante física o de diseño está en `configuracion.py` o en `GeometriaMecanica`.

Inmutabilidad y validación temprana
Un diseño inválido falla al construirse con un mensaje claro, no produce resultados absurdos más tarde.

Funciones pequeñas y puras
Las leyes físicas no tienen efectos secundarios y aceptan escalares o arreglos, lo que simplifica las pruebas y las gráficas.

Responsabilidad única
Calcular, describir piezas, mallar, exportar y presentar están en módulos distintos.

Abierto a extensión
Nuevas gráficas, piezas o comandos se agregan sin modificar el código existente.

7 Pruebas automáticas

Las pruebas (`tests/`, 67 casos con `pytest`) verifican física, reglas de diseño y productos generados:

Archivo	Qué comprueba
`test_fisica.py`	Campo en la cara del imán; límite de dipolo a gran distancia; decrecimiento monótono; campo medio igual a la integral numérica; frecuencia eléctrica; rectificador sin tensión negativa; rizado.
`test_dominio.py`	Resistencia interna calculada a mano; ocupación fabricable; rechazo de polos impares, imanes solapados, demasiadas espiras y parámetros inválidos; salida DC segura para condensador y LED en todo el rango de RPM.
`test_simulador.py`	$\mathcal{E} = -N\,\mathrm{d}\Phi/\mathrm{d}t$ comparando con la derivada numérica; tensión proporcional a la velocidad; máxima transferencia con $R_L = R_{int}$; balance de energía y par de Lenz; sin giro no hay tensión ni par.
`test_cad.py`	Cada pieza es un sólido cerrado y su volumen coincide con el analítico; formato STL binario correcto; alojamientos en las posiciones del diseño.
`test_microred.py`	Potencia máxima con V_{RMS}; arranque del controlador; límites de corriente y de absorción; batería llena sin carga; balance de energía del generador; velocidad de equilibrio (incluida la imposible); corte y reconexión con histéresis; corriente del inversor y validaciones de los componentes.
`test_modelo_realista.py`	Modelo dinámico (Documento 5): solape imán–espira, inductancia, inercia, MPPT y rizado, batería de plomo-ácido, inversor, equilibrio de pares, conservación de la energía, frenado de Lenz, etapas de carga, histéresis y temperatura.
`test_reporte_y_cli.py`	Coherencia reporte–simulador; claves válidas como macros LaTeX; ejecución de cada subcomando de la CLI.
`test_paridad_visualizador.py`	Que `js/config.js` coincida con `configuracion.py` y que `fisica.js` dé los mismos resultados que Python (tolerancia 10^{-12}), y que `dinamica.js` reproduzca `SimulacionDinamica` paso a paso en cuatro guiones (tolerancia 10^{-9}). Se omite si no hay Node.

8 Cómo usar el proyecto

Paso 1 · Instalar dependencias

Se necesita `uv` (gestor de proyectos Python). Desde la raíz del proyecto:

```
uv sync
```

Crea el entorno `.venv` con NumPy, matplotlib y pytest.

Paso 2 · Generar gráficas, STL y valores para los documentos

```
uv run generador-axial simular    # resumen + simulacion/graficas/*.png
uv run generador-axial cad        # cad_3d/stl/*.stl
uv run generador-axial reporte    # docs/latex/generado/*.tex
uv run generador-axial todo       # los tres anteriores
```

Los scripts antiguos siguen funcionando: `python simulacion/modelo_analitico.py` y `python cad_3d/generar_modelos_stl.py`.

Paso 3 · Ejecutar las pruebas

```
uv run pytest
```

Paso 4 · Abrir el visualizador

```
uv run python visualizador_3d/server.py            # ventana propia de 1360x860
uv run python visualizador_3d/server.py --ventana 1600x1000
uv run python visualizador_3d/server.py --en-pestana  # pestaña del navegador
uv run python visualizador_3d/server.py --puerto 9000 --sin-navegador
```

El servidor abre el simulador como ventana de aplicación de Edge, Chrome, Chromium o Brave (sin pestañas ni barra de direcciones) con un perfil propio, para que se respete el tamaño. Si no encuentra ninguno, abre una ventana del navegador predeterminado. Desde una pestaña, el botón "Ventana aparte" abre la misma vista en una ventana emergente. También se puede abrir `visualizador_3d/index.html` directamente con doble clic.

Paso 5 · Recompilar los PDF

```
./docs/compilar_pdf.sh                       # los cinco documentos
./docs/compilar_pdf.sh 02_memoria_de_calculo # solo uno
```

El script ejecuta `generador-axial todo`, compila cada documento dos veces con `pdflatex` (para el índice) en `docs/latex/build/` y copia los PDF a `docs/`. Requiere una distribución TEX con `circuitikz`, `tcolorbox` y `siunitx`.

9 Cómo extender el proyecto

Cambiar el diseño (por ejemplo, más espiras o un imán N52)

1. Edite `diseno_por_defecto()` en `src/generador_axial/configuracion.py`.
2. Replique el cambio en `visualizador_3d/js/config.js`.
3. Ejecute `uv run pytest`: si el diseño es infabricable (por ejemplo, la bobina no cabe) fallará con `ErrorDeDiseno`; si olvidó el paso 2, fallará la prueba de paridad.
4. Ejecute `./docs/compilar_pdf.sh`: gráficas, STL y todos los números de los PDF se actualizan.

Agregar una gráfica

Cree una subclase de `Grafica` en `graficas/catalogo.py` con `nombre_archivo` y `dibujar(figura)`, y añádala a `graficas_estandar`.

Agregar una pieza imprimible

Escriba una función que devuelva una `Pieza` con una `PlacaConAlojamientos` o un `SolidoDeRevolucion` y añádala a `catalogo_de_piezas`. Si necesita una forma nueva, defina su especificación y registre un constructor con `@construir_malla.register`. La prueba de hermeticidad la cubrirá automáticamente.

Agregar un subcomando

Cree una subclase de `Comando` con `nombre`, `ayuda`, `configurar` y `ejecutar`, y añádala en `construir_comandos`.

10 Registro de correcciones

Tabla 2: Principales problemas encontrados en la revisión y su corrección.

Área	Antes	Después
Arquitectura	Constantes y fórmulas duplicadas en la simulación, el CAD, el visualizador y los documentos, con valores distintos.	Diseño único en `configuracion.py`; paridad con JavaScript verificada por prueba; documentos generados.
Física	El flujo se calculaba con el campo en un solo punto.	Campo promediado exactamente sobre el espesor de la bobina, $\bar{B} = 0.1545$ T.
Teoría	Citas a secciones de Sadiku que no correspondían al tema.	Citas corregidas (magnetización 8.5–8.6, fem de transformador y de movimiento 9.3, Gauss 7.5).
Electrónica	Condensador de 25 V y resistencia de LED de 1 kΩ ¼ W, insuficientes a altas RPM.	220 μF × 50 V y 2200 Ω ½ W; una prueba verifica la seguridad en todo el rango.
Ensayos	Tabla de valores teóricos escrita a mano e inconsistente con el modelo.	Tabla generada por `generador-axial reporte`.
CAD	Carrete de una sola pieza (la bobina no se podía extraer); cotas no ligadas al diseño.	Carrete en dos piezas; discos con alojamientos derivados del diseño; mallas cerradas verificadas.
Visualizador	Código monolítico; la pausa no congelaba el tiempo; geometría distinta del diseño; líneas de campo fijas al cambiar el entrehierro; dependencia de CDN.	MVC + Observer; pausa real; geometría coherente con el CAD; líneas reconstruidas; librerías locales; osciloscopio sin *aliasing*.
Gráficas	Curvas superpuestas por coincidencia de escalas (tensión DC y frecuencia).	Frecuencia en un eje secundario superior.
Herramientas	Scripts sueltos sin dependencias declaradas ni pruebas.	Proyecto `uv` con `pyproject.toml`, comando `generador-axial` y 67 pruebas.
Micro-red: potencia	Se usaba $V_{DC}^2/(4R_{int})$, que duplica la potencia disponible.	$P_{max} = V_{RMS}^2/(4R_{int})$ con controlador MPPT y límites de corriente y absorción.
Micro-red: protección	Sin corte por baja tensión: el inversor daba 110 V con la batería vacía.	Corte al 20 % y reconexión al 25 % en Python y JavaScript.
Micro-red: balance	El balance ignoraba el consumo del inversor; constantes fijas (10.5/14.4 V, 500 RPM, 110 V/60 Hz) repetidas en el código.	Balance completo con velocidad de equilibrio (1704 RPM para 15 W); parámetros en `configuracion.py` y `config.js`.
Micro-red: documentos	Citas a Sadiku inexistentes (Cap. 13 y 15) y una fórmula de carga distinta de la del código.	Transformador citado en Sec. 8.8 y 9.3; SPWM como teoría de circuitos; fórmulas iguales al modelo.
Micro-red: visualizador	SoC guardado en el estado observable en cada cuadro; la traza de 220 V se recortaba en el osciloscopio; código muerto (`CargaAc`).	Clase `MicroRed` fuera del estado; escala automática de 200 V/div; código muerto eliminado.